

為什麼現在要導入 Agent 與 Git ?

Agent + Git + Security | Team Onboarding Brief

工程成果與決策分散

需要共同入口

不清楚從哪裡開始

需要可認領工作包

Agent 能行動，也會帶來風險

需要受控 workflow

今天會講什麼

先建立治理共識，再帶到實際 Onboarding。



01 Agent、Git 與安全

能力、權限、風險

02 工具與 Repository Quick Start

先從唯讀任務開始

03 5G SDR 案例與工作包

用真實專題流程說明

04 驗收標準與下一步

確認第一個安全任務

✓ 核心 / 必做

⚠ 風險 / 禁止

i 補充 / 待確認

■ 背景 / 次要

Agent 和一般聊天 AI 有什麼不同

真正的差異，在於能不能對外部環境採取行動。



TEXT OUTPUT

一般聊天 AI

- 回答問題
- 主要輸出文字
- 通常不直接改變環境

TOOL ACTIONS

Agent

- 完成任務
- 讀檔、使用工具、執行命令
- 可能產生實際變更

LLM + Tools + Permissions + Task Loop = Agent

CIA 風險基礎

機密性 (Confidentiality) — 可能讀取、輸出或外傳不應存取的資料

完整性 (Integrity) — 可能修改程式、設定、文件或證據

可用性 (Availability) — 可能中斷服務、耗盡資源或影響設備運作

Agent 能接觸哪些資源

風險大小取決於 Harness 實際給了哪些工具與權限。



FILES

檔案

讀取與修改 | CIA : C · I

SHELL

命令列

執行與刪除 | CIA : I · A

NETWORK

網路

外部服務與遠端主機 | CIA : C
· A

CREDENTIALS

憑證

Token、SSH
Key、密碼 | CIA : C · I · A

共同安全控制

最小權限 (Least Privilege)

預設拒絕 (Deny by Default)

職責分離 (Separation of
Duties)

可稽核性 (Auditability)

一次受控 Agent 任務的生命週期

Agent 任務從 Scope、權限與 Stop Point 開始。



先界定，再執行；先看證據，再放行。

Git 能控制什麼、不能控制什麼

Git 是變更控制與稽核介面，不是作業系統 Sandbox。

Git 可以提供

- Branch | 隔離工作
- Diff | 顯示修改
- Commit | 保存歷史
- Pull Request | 人工審查
- Revert | 回復 Git 內容

Git 無法直接阻止

- Agent 讀取 Secret
- 資料傳至外部網路
- 修改 Repo 外檔案
- 執行破壞性 Shell
- 操作正式設備或 Runtime

Git is a Control Surface, not a Sandbox.

Agent 資安與專用 SSH Key 案例

專用 Key 能分離登入憑證，但不會自動降低 Linux 帳號權限。



KEY | 身分驗證 (Authentication)

你是誰？
SSH Key 驗證登入身分

LOCK | 權限授予 (Authorization)

你可以做什麼？
Linux 帳號、檔案與命令權限

DOC ✓ | 來源權威性 (Source Authority)

哪份資料可作為判斷依據？
經驗證的紀錄、正式狀態文件或核准決策

本簡報中的 **Authority** 指來源在特定判斷中的證據或決策優先性，不是身分驗證，也不是系統權限。

三種 Agent 工具的共同 Quick Start

介面不同，但第一個任務都應受限、唯讀、可驗證。



Local Model · Windows · Read-only First

Codex Local Agent

Codex App / CLI + Ollama

使用本地模型盤點一份指定
Markdown

CLI Agent

Gemini CLI

唯讀盤點指定目錄

\$ gemini

Repository Agent

Claude Code

檢查 Git 狀態並提出建議

\$ claude

01

啟動工具

02

進入 Repository

03

確認 Branch /
Status

04

讀取
AGENTS.md

05

執行 Read-only
Task

06

再次確認 Git
Status

完整 Windows 安裝、連接與驗證流程：附錄 A

Agent 介面、Ollama Server 與 Model 是不同層

工具路徑與推論路徑分開；Local Model 不會直接取得 Repository 或 Shell。



Agent 介面負責任務與工具；Ollama 負責推論服務；Model 負責生成與推理。

如何取得並理解 Repository

第一次 Clone 與日常同步，是兩種不同情境。



第一次加入

- 1 Clone Repository
複製儲存庫
- 2 Enter Repository
進入專案目錄
- 3 Check Branch / Status
確認分支與狀態
- 4 Read Entry Documents
閱讀入口文件
- 5 Run Read-only Task
執行唯讀任務

已經有 Repo

- 1 Check Working Tree
檢查工作目錄
- 2 Confirm Branch
確認目前分支
- 3 Synchronize When Safe
確認安全後再同步
- 4 Read Current Progress
閱讀目前進度
- 5 Select Task
選擇任務

執行 Pull 前，先確認本機 Working Tree 沒有未處理的變更。

如何使用 5g-sdr-ops Agent Harness

不只要知道讀哪些文件，也要知道每份文件能證明什麼。



PROGRESS.md 更新或產生新進度後，於 5G SDR | Lab Operations 群組同步摘要。

5G SDR 的真實工作案例

Lab01 Recovery 透過盤點、Gate、驗證與 Git 紀錄完成收束。



01

Read-only Inventory
唯讀盤點
R0 / R1 狀態盤點與權威比對

02

Backup & Authorization Gates
備份與授權閘門
Phase 4A 備份驗證；Phase
4B 明確授權

03

**Controlled Runtime
Validation**
受控運行驗證
EPC、eNB、UE、ZeroMQ、A
ttach 與 ICMP

04

Controlled Shutdown
受控關閉
依序停止服務並確認收束

05

Progress + PR + Merge
進度紀錄、PR 與合併
PROGRESS.md、Commit 與
Pull Request

Governance / 治理結果

Phase 4A–4C PASS / CLOSED
runtime closeout commit fb4f195
presentation merge commit 3387ce0

Runtime Evidence / 運行證據

EPC / eNB / UE initialization success
ZeroMQ 2000 / 2001 PASS
Cell Search → RA → RRC Connected →
Attach
UE IP 172.16.0.2

Validation / 驗證結果

Bidirectional ICMP : 0% packet loss
Controlled shutdown PASS

流程證據摘要

受控運行驗證涵蓋 EPC、eNB、UE、ZeroMQ、Attach 與
ICMP；完成後依序停止服務並確認收束。

完成宣稱邊界

NAT / Internet、Throughput、Wireshark / PCAP、URLLC 與 MBMS
功能不在本次完成宣稱內。

暑假方向與可認領工作包

不必一次負責整條研究線，可以先交付一個明確的小任務。



環境維護

L0

第一個任務

建立版本與設備 Inventory

GitHub 文件

L0-L1

第一個任務

盤點一份過期文件

Lab02 / 03 / 04

L0-L2

第一個任務

整理前置條件與成功標準

可觀測性

L0-L2

第一個任務

建立 Log Source Inventory

課堂教材

L1

第一個任務

製作一頁 Quick Start

建議先從 L0-L2 開始；L3-L4 需另行授權與共同操作。

做到什麼程度，能夠交給我 Review ？



達成事前約定的 Minimum Done，就可以提交 Review。

Minimum Done

可以提交 Review

- ✓ 完成原定 Scope
- ✓ 交付物與證據齊全
- ✓ 沒有已知重大錯誤

Target Quality

團隊正常期待

- 結構與格式清楚
- 主要判斷有證據
- 他人可以接續工作

Stretch Optional

加分，不阻塞

- 更多圖表或測試
- 自動化與延伸分析
- 完整排版或教材化

Review 後新想到的改善，預設屬下一輪，不回溯性提高原任務的最低完成條件。

目前 Review 標準 | ACP : PARKING / OBSERVATION

ACP 快照 | 固定本輪標準，阻止 Scope Creep

ACP 不降低品質；它避免 Review 當下的新靈感自動變成本輪的必要條件。



沒有校準時

完成原需求
↓
再改一點
↓
這裡也順便改
↓
乾脆一起重構

⚠ Minimum Done 消失

任務無法交付
Review 時間膨脹
完成標準持續上升

ACP 的校準閘門

Review 時出現新的改善想法

這個問題是否代表原需求或成果本身有缺陷？

本輪必須修正 | Need

- ⚠ 原需求描述錯誤
- ⚠ 重大 Bug
- ⚠ 安全問題
- ⚠ Architecture 違反

下一輪 Improvement Nice-to-have

- + 更漂亮的排版
- + 更多延伸分析
- + 額外自動化
- + 順便重構其他區域

Reviewer 的靈感，不等於新的 Mandatory Requirement。

ACP 校準的是什麼？

不是
Reviewer 能想到多少改善

而是
能否區分 Need 與 Nice-to-have

LIFECYCLE
Parking → Observation
→ Draft v0.1 → Pilot
→ Protocol v1.0

目前：PARKING / OBSERVATION

本次需要確認的決策與下一步



讓每個人離開會議前，都知道自己的第一個安全任務。

- 1 每位參與者先使用哪一套 Agent 工具？
- 2 每位參與者認領哪一個小型工作包？
- 3 是否安排獨立 Quick Start 操作時間？
- 4 Git 教學需要到 Branch、PR，還是更進一步？
- 5 是否希望後續發展為正式課堂教材？

暫定底線：Read-only 起步 | 不直接修改 main | Runtime 操作另行協調

先確認每個人的第一個安全任務。

附錄 | Windows : Codex + Ollama 本地 Agent



Windows PowerShell 為主要環境；先確認共同前置，再選 Codex App 或 Codex CLI。

```
Windows

ollama launch codex-app
ollama launch codex-app --model <model-name>
```

官方文件列出 Windows 支援；建議 Context $\geq 64K$ 。

```
Windows PowerShell

npm install -g @openai/codex
codex --oss
codex --oss -m <model-name>
# quick setup: ollama launch codex
```

使用 Ollama 的 OpenAI-compatible 本機服務。

```
先確認環境

ollama --version
ollama list
確認選定模型 Tag 可載入
確認 Codex 已安裝
```

不要在未查證時固定不存在的模型 Tag。



安全提醒：Local Model 不會降低 Codex 權限；Repository Write、Shell、Network 與 Approval 仍由任務邊界控制。

附錄 | 本地模型如何選？

先依硬體與任務選擇候選模型，再用同一個唯讀任務實測，不只比較參數量。



低資源起始

LOW RESOURCE

Qwen3 4B
Gemma 4 E2B / E4B

- ✓ 下載與啟動成本較低
- ✓ 適合單檔唯讀任務
- ⚠ 複雜 Repository 推理與工具使用可能不穩定

平衡選項

BALANCED

Qwen3 8B / 14B
Gemma 4 12B

- ✓ 指令遵循與分析通常較佳
- ⚠ Context 增加後記憶體需求明顯上升
- 依實際速度與錯誤率調整

進階可選

ADVANCED OPTIONAL

Qwen3 30B-A3B
Qwen3-Coder 30B
Gemma 4 26B-A4B / 31B

- ✓ 較複雜的 Coding / Agent 任務
- ⚠ 權重、Context 與等待成本增加
- ⚠ MoE 仍需載入完整權重

[CHIP] 權重 / 量化
權重 = 已學參數；量化 = 較低精度保存
低精度通常更省空間，也可能影響品質 / 穩定性

[MEM] RAM / VRAM
RAM = 系統記憶體；VRAM = GPU 專用
更多層放進 VRAM 通常較快；不足會轉 RAM / CPU

[DOC] Context
一次任務的文字、對話與檔案內容
越長通常越吃記憶體與處理時間

[TIME] 速度
包含首次回應等待與持續生成速度
較大模型或較長 Context 通常更慢

[中/<>] 中文 / 程式理解
繁中、程式碼與專業名詞能力各異
不能只看參數量，要用相同任務比較

[TOOL] 工具呼叫
能否穩定產生符合格式的工具操作
檢查 Scope、參數格式與停止條件

同一個唯讀任務評估 | 選候選 → 確認可載入 → 相同 Read-only Task → 記錄速度 / 記憶體 / 錯誤 → 檢查遵循 → 升 / 降級

模型下載大小不等於實際 RAM / VRAM 用量；量化、Context、GPU Offload 與任務內容都會影響實際需求。

附錄 B | Git 最小指令表



只保留支撐 Branch、Diff、Review 與 PR 的入門流程。

第一次取得 Repo

```
PowerShell  
git clone <repo-url>
```

加入與提交

```
PowerShell  
git add <files>  
git commit -m "docs: ..."
```

查看狀態

```
PowerShell  
git status --short --branch
```

推送分支

```
PowerShell  
git push -u origin <branch>
```

建立工作分支

```
PowerShell  
git switch -c docs/<task-name>
```

建立 PR

```
PowerShell  
gh pr create
```

檢查修改

```
PowerShell  
git diff
```

入門流程不包含：**git reset --hard**、**git clean**、**強制推送或直接修改 main**。

附錄 B2 | Git 例外處理 | Diverged main 與 Squash Merge



以下不是日常入門指令，只在前置條件全部成立時使用。

Case 1 | main 無法 fast-forward

前置：main | Working Tree clean
origin/main 已確認為權威
本機額外 Commit 不需成為正式歷史

Case 2 | Squash Merge cleanup

Squash 在 main 建立新的單一 Commit；
原分支 Commit 未必成為 main 的祖先，
因此 branch -d 可能顯示 not fully merged。

PowerShell | 先比對再備份

```
git switch main
git status --short --branch
git fetch origin
git log --oneline --left-right main...origin/main
$stamp = Get-Date -Format yyyyMMdd-HH:mm
git branch "backup/main-before-realign-$stamp"
git reset --hard origin/main
git status --short --branch
```

PowerShell | 先驗證 PR

```
git fetch --prune origin
git switch main
git pull --ff-only
gh pr view <PR-number> --json state,mergedAt,mergeCommit,url
git branch -D <branch>
git branch -r --list "origin/<branch>"
# 僅在遠端 branch 仍存在時執行
git push origin --delete <branch>
```

為什麼每一步都需要

fetch：只更新遠端追蹤資訊
log：看兩側各自多出的 Commit
backup：保留本機舊指標
reset：精確對齊已確認的權威遠端

刪除前的證據

PR state = MERGED；Base 正確
main 已含 Squash Commit
分支沒有唯一的本機成果
-D：確認後刪本機；--delete：遠端仍存在才用

⚠ 受控例外

只有 Working Tree clean、遠端權威已確認且已建 Backup Branch，才可
reset --hard。
禁止 Force Push main、直接 Merge 或跳過 log 比對。

⚠ 不得用 -D 略過尚未 Review、尚未 Merge，或仍包含唯一工作成果的 branch。

日常流程仍使用 pull --ff-only 與 branch -d；reset --hard 和 branch -D 僅屬受控例外處理。

附錄 C | Level 0–4 任務風險與授權



等級愈高，愈需要明確批准、共同操作與證據。

L0	唯讀整理	摘要、版本表、術語表	Read-only
L1	Repo 文件修改	README、Checklist、圖表	Branch + Diff
L2	分析與設計	Flow、Evidence Matrix	Review Gate
L3	受控環境驗證	Log 分析、設定比較	Approval Required
L4	Runtime / 設備操作	啟動服務、修改設定、USRP	Joint Operation

本次建議：先從 L0–L2 開始；L3–L4 由任務包與現場協調另行授權。

附錄 D | 5G SDR 三平面網路拓樸



真實架構可用於內部理解，但不代表現場重新啟動 Runtime。

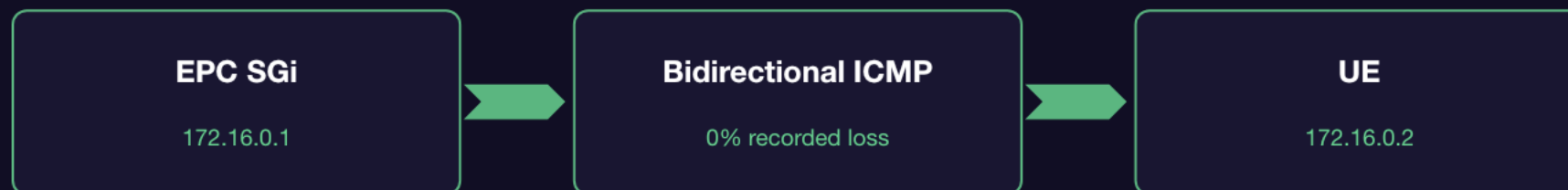
Management Plane



Sample Plane



User Plane



附錄 E | 高風險操作與批准證據



Human-in-the-loop 必須能看懂即將發生的操作。

SSH / Remote Host

sudo

修改設定檔

執行未知 Script

使用外部網路

接觸 Secret

建立 Commit / PR

啟動 Runtime

APPROVAL PACKET

Task Scope

Affected Files

Command Plan

Network Usage

Credential Contact

Diff

Validation Result

Rollback Method

批准不是按下 Allow；批准是理解範圍、指令、證據與回退方式。



THANKS

有問題，或想先認領哪一個工作包？

5G SDR Agent + Git + Security | Team Onboarding Brief

Design attribution

Visual language adapted from “Technology Market Research Pitch Deck” by Slidesgo. Icons / graphics originally credited by Slidesgo to Flaticon and Freepik. This modified presentation retains attribution pending license-tier confirmation.

5G SDR | Engineering Onboarding Brief

